

Precinct-Level Demand Rates: Analysis & Downstream Trace

Part 1: What the Visualization Shows

The Chart

The bar chart titled “**Precinct-Level Demand Rates (Red=High, Green=Low)**” displays the average number of motor vehicle crashes per day for each of Manhattan’s 30 police precincts, sorted from highest to lowest.

Key Observations

Metric	Value
Highest demand	Precinct 19 (~8.3 crashes/day)
Second highest	Precinct 18 (~7.2 crashes/day)
Median	2.9 crashes/day (orange dashed line)
Lowest demand	Precinct 114 (~0.05 crashes/day)
Demand ratio	Top precinct has ~166x the demand of the lowest

Color Coding

- **Red (Crimson):** High-demand precincts ($\geq$ 75th percentile of `lambda_per_hour`) — Pcts 19, 18, 14, 1, 17, 13, 10
- **Blue (Steel Blue):** Medium-demand precincts (between 25th and 75th percentile)
- **Green (Forest Green):** Low-demand precincts ($\leq$ 25th percentile) — Pcts 28, 30, 26, 22, 50, 52, 114

What “Crashes per Day” Represents

Each bar shows `lambda_per_day` — the average arrival rate of crashes in that precinct, computed as:

$$\text{lambda_per_day} = (\text{total crashes in precinct}) / (\text{total hours in dataset}) \times 24$$

This is the Poisson intensity parameter (λ) for each precinct, expressing the expected number of crash-related EMS calls per day.

Part 2: How It's Generated

Source Code

File: `scripts/demand_modeling.py`, lines 283–518

Data Flow

1. **Raw data:** `data/processed/crashes_manhattan.parquet` (416,434 crash records, 2012–2026)
2. **Spatial join:** Crash locations (lat/lon) are joined with `data/processed/precincts_manhattan.geojson` using GeoPandas `sjoin` to assign each crash to a precinct
3. **Aggregation:** Crashes are counted per precinct and divided by total hours to get `lambda_per_hour`
4. **Output:** Saved to `data/processed/demand_lambda_precinct.csv` with columns:
 - `precinct` — precinct ID
 - `total_crashes` — raw count
 - `crash_rate_per_hour` — λ (crashes/hour)
 - `demand_weight` — normalized proportion of total demand

Quantile Classification

```
q75 = precinct_df['lambda_per_hour'].quantile(0.75) # High-demand threshold
q25 = precinct_df['lambda_per_hour'].quantile(0.25) # Low-demand threshold
```

Part 3: Downstream Usage — Complete Trace

3.1 Optimization Models (Most Critical Usage)

File: `src/ems_readiness/optimization/allocator.py`, lines 115–116

```
dl = pd.read_csv("data/processed/demand_lambda_precinct.csv")
demand = dl.set_index(dl["precinct"].astype(str))["crash_rate_per_hour"]
```

The `crash_rate_per_hour` column becomes the **demand weight** (`dj`) in all optimization formulations:

Demand-Weighted Allocation (P2) — Primary Optimized Policy

File: `src/ems_readiness/optimization/models.py`, line 66–69

Objective: $\min \sum_j d_j \times \sum_i t_{ij} \times y_{ij}$

Where `dj` is the precinct's crash rate. **This means high-demand precincts like Pct 19 (8.3 crashes/day) have ~166x more influence on where units are placed than Pct 114.** The optimizer concentrates units near Midtown/Upper East Side where crash density is highest.

P-Median Model

Same demand weights are used to minimize total demand-weighted distance — firehouses serving high-demand precincts are prioritized.

Maximal Coverage Model

Demand weights determine which precincts matter most for coverage — covering Pct 19 contributes far more to the objective than covering Pct 114.

Demand-Proportional Allocation (P1 Baseline)

File: `src/ems_readiness/optimization/policies.py`, lines 54–100

Each firehouse gets units proportional to the demand of its nearest precincts. Firehouses near Pcts 19, 18, 14 get the most units.

Uniform Allocation (P0 Baseline)

P0 **ignores** demand entirely — distributes K units evenly across firehouses. This is exactly why the visualization matters: it shows the spatial heterogeneity that P0 fails to account for.

3.2 Simulation — Arrival Generation (NHPP)

File: `src/ems_readiness/demand/arrival_generator.py`, lines 102–118

```
precinct_rates = dict(
    zip(precinct["precinct"].astype(int), precinct["crash_rate_per_hour"])
)
```

The precinct rates are used for **spatial allocation of simulated incidents**:

1. The NHPP generates arrivals at the Manhattan-wide rate: $\lambda(t) = \text{base_rate} \times \text{hourly_factor} \times \text{dow_factor}$
2. Each arrival is assigned to a precinct with probability proportional to `crash_rate_per_hour`:

```
python
# Lines 91-96: Probability of incident in precinct p
prob_p = precinct_rates[p] / sum(precinct_rates.values())
```
3. This means $\sim 8.3/83.6 \approx 10\%$ of simulated incidents go to Pct 19, while only $\sim 0.05/83.6 \approx 0.06\%$ go to Pct 114

File: `src/ems_readiness/simulation/engine.py`, line 241

```
precinct = int(row.get("precinct", 1))
```

The precinct assignment drives **dispatch decisions** — the dispatcher finds the nearest available unit to the incident's precinct using the travel-time matrix.

3.3 Dispatch & Response Time Evaluation

File: `src/ems_readiness/simulation/dispatcher.py`

When an incident arrives in precinct `p`, the dispatcher looks up `travel_time_matrix[firehouse, p]` for all available units and sends the nearest one. Because P2 places more units near high-demand precincts, those precincts get faster response times under P2 than under P0.

File: `src/ems_readiness/optimization/allocator.py`, lines 245–258

```
def _evaluate_response_time(self, allocation):
    min_tt = tt_sub.min(axis=0) # nearest active firehouse per precinct
    weighted = (min_tt * self.demand[precincts]).sum() # demand-weighted
```

Response time evaluation is also demand-weighted — improvements in high-demand precincts matter more.

Part 4: Impact on Results — P0 vs P2

Why Spatial Heterogeneity Matters

The 166:1 ratio between highest and lowest demand precincts means:

- **P0 (Uniform)** wastes capacity by placing the same number of units in low-demand areas (Pct 114, 52, 50) as in high-demand areas (Pct 19, 18, 14)
- **P2 (Demand-Weighted)** concentrates resources where crashes actually happen, dramatically reducing response times for the majority of incidents

Specific Examples

Precinct	Crashes/Day	P0 Treatment	P2 Treatment
Pct 19 (Midtown South)	8.3	Same as everywhere	Gets more nearby units; nearest fire-house heavily staffed
Pct 18 (Midtown North)	7.2	Same as everywhere	Prioritized by optimizer
Pct 114 (Central Park)	0.05	Gets units it barely needs	May get zero dedicated units; served by nearby Midtown units

The Validation Pilots Confirm This

From the project's verification & validation:

- **Pilot 1:** P0 vs P2 directional comparison → **P2 dominates** (lower demand-weighted response time)
 - **Pilot 2:** Response time decreases monotonically with fleet size → more units help, but P2 deploys them smarter
 - **Pilot 3:** Response time increases with demand intensity → high-demand precincts benefit most from optimized placement
-

Part 5: Complete Data Flow Diagram

Raw Crash Data (2.2M NYC records)

[scripts/demand_modeling.py §2] Spatial **join** with precinct polygons

[scripts/demand_modeling.py §5] **Aggregate**: crashes/hour per precinct

demand_lambda_precinct.csv
(precinct, crash_rate_per_hour, demand_weight)

Optimization (allocator.py)

demand = dl["crash_rate_per_hour"]

P2: build_demand_weighted() $\min \sum d_j \cdot t_{ij}$

P-Median: build_p_median()

Coverage: build_maximal_coverage()

P1: demand_proportional_allocation()

Simulation (arrival_generator.py)

precinct_rates = {"precinct": crash_rate_per_hour}

NHPP spatial allocation $P(\text{precinct}=p) \propto \lambda_p$

engine.py dispatch by precinct

Response time by precinct

Visualization (this chart)

fig_precinct_demand.png

Communicates spatial heterogeneity
to justify demand-weighted optimization

Part 6: Key Code References

Component	File	Key Lines	Role
Visualization	scripts/de-mand_modeling.py	496–518	Creates the bar chart
Data generation	scripts/de-mand_modeling.py	283–312	Computes precinct λ values
CSV output	scripts/de-mand_modeling.py	392	Saves de-mand_lambda_precinct.csv
Optimization loading	src/ems_readiness/optimization/allocator.py	115–116	Reads demand as crash_rate_per_hour
Demand-weighted model	src/ems_readiness/optimization/models.py	30–83	Uses demand as objective weight
P-median model	src/ems_readiness/optimization/models.py	86+	Uses demand as distance weight
Simulation loading	src/ems_readiness/demand/arrival_generator.py	110–111	Reads crash_rate_per_hour for spatial allocation
Spatial arrival allocation	src/ems_readiness/demand/arrival_generator.py	186–189	Assigns incidents to precincts proportionally
Dispatch	src/ems_readiness/simulation/dispatcher.py	63–107	Routes units to incident precincts
Config	configs/demand.yaml	—	Base rate (3.48/hr), lambda table paths
Config	configs/optimization.yaml	—	K values, capacity, coverage threshold